

22

Image Classification

The most common computer vision task

Give a whole image one label — cat, dog, or defect

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it does	Looks at an image and gives it one label
Output	Probabilities for each possible class
Main tool	A CNN, usually pre-trained
Your project	Product Image Classifier (ResNet18, 99.3%)
Why learn it	The foundation of all vision tasks

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is Image Classification?	The task and how it works
2	Three Kinds	Binary, multi-class, multi-label
3	How a Model Decides	From image to probabilities
4	The Full Workflow	Data to trained model
5	Folders for Data	How to organise images
6	Training and Watching It Learn	Reading the curves
7	Measuring Success	Accuracy and the confusion matrix
8	Making a Prediction	Using the trained model
9	Tips for Better Accuracy	Practical boosts
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is Image Classification?

Image classification is the simplest and most common computer vision task. You give the model one image, and it gives back one label that says what the image is. 'This is a cat.' 'This is a shirt.' 'This product has a defect.' One picture in, one answer out.

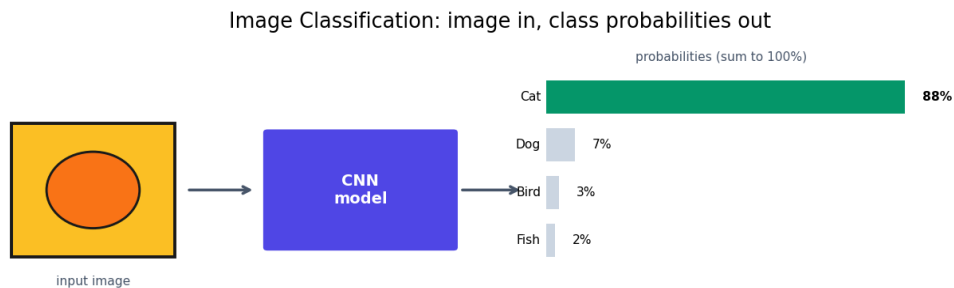


Fig 1 — An image goes into the model, and out come probabilities. The highest one is the answer.

The model doesn't just say 'cat'. It gives a probability for every possible class — like 88% cat, 7% dog, 3% bird, 2% fish. These add up to 100%. The class with the highest probability is the final prediction. This is exactly what your Product Image Classifier does, sorting fashion products into categories.

■ *This is the task behind your Product Image Classifier project (ResNet18, 99.3% accuracy). This topic is a focused deep dive into doing it really well.*

2. Three Kinds of Classification

Image classification comes in three forms, depending on how many classes there are and how many you pick.

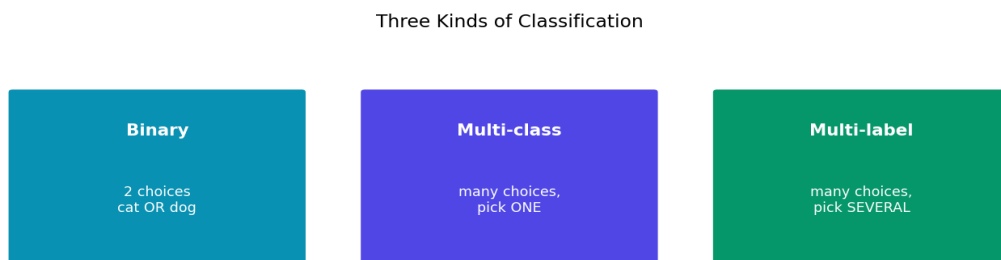


Fig 2 — Binary (2 options), multi-class (pick one of many), multi-label (pick several).

Kind	Meaning	Example
Binary	Only two classes	Defect or no defect
Multi-class	Many classes, pick exactly ONE	Cat, dog, bird, or fish
Multi-label	Many classes, pick SEVERAL	A photo tagged 'beach' AND 'sunset'

Most projects are multi-class, like sorting products into categories. Binary is the special case of just two options. Multi-label is when an image can belong to more than one class at the same time. The difference mainly affects the final layer and the loss function you choose.

3. How a Model Decides

Inside, the CNN finds features (edges, shapes, textures, objects) as you saw in the CNN topic. The final layers take all those features and turn them into a score for each class. Then a softmax function turns those scores into probabilities that add up to 100%.

```
# The final part of a classification model
# 'features' is what the CNN found in the image

scores = final_layer(features)  # one raw score per class
# e.g. [3.2, 0.8, -1.0, 0.1]

probs = softmax(scores)        # turn into probabilities
# e.g. [0.88, 0.07, 0.02, 0.03] (sum = 1.0)

prediction = probs.argmax()     # pick the highest -> class 0
```

■ *The probabilities are useful beyond just the answer. A 95% top score means the model is confident. A 40% top score means it's unsure — useful to know when you want to flag uncertain cases for a human to check.*

4. The Full Workflow

Building an image classifier follows a clear set of steps. You've seen most of these in earlier topics; here they come together for this one task.

Step	What you do
1. Collect images	Gather labelled images, one label per image
2. Split	Training set, validation set, test set
3. Preprocess	Resize and normalize every image
4. Augment	Flip, rotate, crop the training images
5. Pick a model	A pre-trained CNN (ResNet, EfficientNet)
6. Swap final layer	Match it to your number of classes
7. Train	Run the training loop, watch the curves
8. Evaluate	Check accuracy on the test set
9. Predict	Use it on brand new images

5. Folders for Data

The easiest way to organise image data is one folder per class. Put all cat images in a 'cat' folder, all dogs in a 'dog' folder, and so on. PyTorch can read this structure automatically and figure out the labels from the folder names.

```
data/  
  train/  
    cat/  cat1.jpg, cat2.jpg, ...  
    dog/  dog1.jpg, dog2.jpg, ...  
    bird/ bird1.jpg, ...  
  val/  
    cat/  ...  
    dog/  ...  
  
# PyTorch reads it automatically:  
from torchvision import datasets  
train_data = datasets.ImageFolder('data/train', transform=tf)  
print(train_data.classes)  # ['bird', 'cat', 'dog']
```

■ *Keep your classes balanced when you can — roughly the same number of images per class. If one class has way more images, the model can become biased toward it.*

6. Training and Watching It Learn

During training, you watch two numbers: training accuracy and validation accuracy. Plotting them tells you how learning is going and warns you about overfitting.

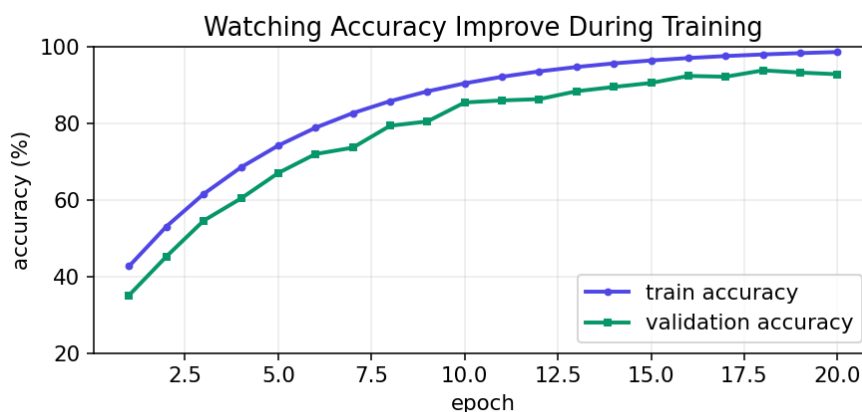


Fig 3 — Both accuracies rising together is healthy. A big gap means the model is overfitting.

If both go up and stay close, great — the model is learning real patterns. If training accuracy keeps climbing but validation accuracy stalls or drops, the model is memorising the training images instead of learning. That's overfitting, and it means you should add augmentation, get more data, or stop training earlier.

7. Measuring Success

Accuracy is the headline number: out of all test images, how many did the model label correctly. But for more than two classes, the confusion matrix shows you exactly where it gets confused.

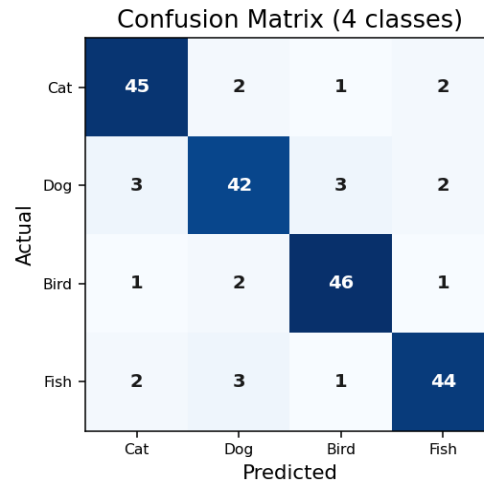


Fig 4 — A confusion matrix. The strong diagonal means most predictions are correct.

Read the confusion matrix like this: each row is the true class, each column is what the model guessed. The diagonal (top-left to bottom-right) is correct answers. Off-diagonal numbers are mistakes. If you see 'cat' often guessed as 'dog', you know those two classes look similar to the model and may need more or better training images.

```
from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_true, y_pred))
print(classification_report(y_true, y_pred))
# gives precision, recall, f1 for each class
```

8. Making a Prediction

Once trained, using the model on a new image is just a few lines: load the image, preprocess it the same way as training, run it through, and read the top class.

```
from PIL import Image
import torch

model.eval()  # switch to prediction mode

img = Image.open('new_photo.jpg')
x = preprocess(img).unsqueeze(0)  # prepare + add batch dim

with torch.no_grad():
    output = model(x)
    probs = torch.softmax(output, dim=1)
    top_class = probs.argmax(1).item()
    confidence = probs.max().item()

print(class_names[top_class], f'{confidence:.0%}')
```

■ *Always preprocess a new image exactly the same way you preprocessed training images — same resize, same normalize. A mismatch here is a very common cause of bad predictions.*

9. Tips for Better Accuracy

- **Use transfer learning** — start from a pre-trained model. Biggest single win.
- **Add data augmentation** — flips, rotations, crops. Free accuracy and less overfitting.
- **Get more data** — more images per class almost always helps.
- **Balance your classes** — similar counts per class avoids bias.
- **Fine-tune gently** — after training the new layer, unfreeze and train the whole model with a tiny learning rate.
- **Use early stopping** — stop when validation accuracy stops improving.
- **Try a bigger model** — if you have enough data, a larger CNN can help.

■ *Transfer learning plus augmentation is the combination that took your Product Image Classifier to 99.3%. These two together are the backbone of almost every strong image classifier.*

10. Common Mistakes

Mistake	Why it's a problem	Fix
Different preprocessing for test	Predictions go wrong	Use the same steps as training
No transfer learning	Wastes data and time	Start from a pre-trained model
Imbalanced classes	Model favours the big class	Balance data or weight the loss
Augmenting test images	Unfair, wrong scores	Augment training only
Ignoring the confusion matrix	Miss which classes confuse	Read it to find weak spots
Forgetting <code>model.eval()</code>	Wrong predictions	Call <code>eval()</code> before predicting

Quick Reference Cheatsheet

Task	Code
Load image folders	<code>datasets.ImageFolder('data/train', tf)</code>
See class names	<code>data.classes</code>
Pre-trained model	<code>models.resnet18(weights='DEFAULT')</code>
Swap final layer	<code>model.fc = nn.Linear(in, n_classes)</code>
Loss (multi-class)	<code>nn.CrossEntropyLoss()</code>
Prediction mode	<code>model.eval()</code>
Get probabilities	<code>torch.softmax(output, dim=1)</code>
Top class	<code>probs.argmax(1)</code>
Confidence	<code>probs.max()</code>

Accuracy	<code>(preds == labels).float().mean()</code>
Confusion matrix	<code>confusion_matrix(y_true, y_pred)</code>
Full report	<code>classification_report(y_true, y_pred)</code>

The one idea to remember: image classification gives a whole image one label, by turning the CNN's features into a probability for each class. Use a pre-trained model, augment your data, and watch both accuracy curves to build a strong classifier.

Next Topic: Object Detection — going beyond one label to finding and boxing many objects in a single image, the task behind YOLO and your planned object detection project.